

Projet Big Data : Analyse des abonnés d'une plateforme de streaming avec Hadoop HDFS et Apache Spark

Informations étudiant

- **Nom** : Zakaria
 - **Prénom** : Bayad
 - **Projet** : Analyse des abonnés d'une plateforme de streaming
-

1. Introduction

L'objectif de ce projet est de mettre en œuvre une architecture Big Data basée sur **Hadoop HDFS** et **Apache Spark** afin d'analyser les données des abonnés d'une plateforme de streaming.

Le jeu de données fourni, `abonnes.csv`, contient les informations des abonnés, notamment :

- l'identifiant de l'abonné ;
- la ville ;
- l'âge ;
- le type d'abonnement ;
- la durée d'utilisation ;
- le montant payé.

Le fichier original contient environ **10 000 lignes** pour une taille d'environ **337 Ko**. Cette taille est suffisante pour réaliser les analyses demandées avec Spark, mais elle reste trop faible pour démontrer clairement le fonctionnement d'un vrai stockage distribué sur plusieurs DataNodes.

En effet, HDFS stocke les fichiers sous forme de blocs. Lorsqu'un fichier est très petit, il est généralement stocké dans un seul bloc, ce qui ne permet pas d'observer une vraie répartition des blocs sur plusieurs DataNodes.

Pour cette raison, un script Python nommé `magnify.py` a été créé afin de générer un fichier plus volumineux à partir du fichier original. Ce fichier, nommé `abonnes_cluster_demo.csv`, atteint environ **22 Mo**. Il sert uniquement à démontrer le fonctionnement du cluster Hadoop avec plusieurs DataNodes.

Deux fichiers ont donc été utilisés :

- `abonnes.csv` : fichier principal utilisé pour répondre aux questions du devoir.
- `abonnes_cluster_demo.csv` : fichier plus volumineux utilisé uniquement pour démontrer la distribution des blocs HDFS sur plusieurs DataNodes.

2. Objectifs du projet

Les objectifs principaux sont :

1. Créer un environnement Hadoop/Spark avec Docker.
 2. Importer le fichier CSV dans HDFS.
 3. Créer un Notebook Spark avec PySpark.
 4. Configurer l'environnement Spark.
 5. Charger les données depuis HDFS.
 6. Exécuter les requêtes Spark demandées.
 7. Analyser les résultats obtenus.
 8. Démontrer le stockage distribué avec un cluster Hadoop composé de plusieurs DataNodes.
-

3. Technologies utilisées

Technologie	Rôle
Docker	Déploiement des services sous forme de conteneurs
Docker Compose	Orchestration des conteneurs
Hadoop HDFS	Stockage distribué des fichiers
NameNode	Gestion des métadonnées HDFS
DataNode	Stockage physique des blocs HDFS
Apache Spark	Traitement distribué des données
PySpark	Utilisation de Spark avec Python
Jupyter Notebook	Interface de développement et d'analyse
Python	Génération du fichier de démonstration
Pandas	Manipulation du fichier CSV dans <code>magnify.py</code>

4. Architecture du projet

4.1 Architecture logique

Fichier CSV

↓

```
HDFS Hadoop
  ↓
Apache Spark
  ↓
Notebook Jupyter
  ↓
Résultats d'analyse
```

4.2 Architecture Hadoop simple

Cette architecture est utilisée pour réaliser le devoir principal.

```
+-----+
|  NameNode  |
+-----+
      |
+-----+
|  DataNode  |
+-----+

+-----+
| Spark Notebook |
+-----+
```

4.3 Architecture cluster Hadoop

Cette architecture est utilisée pour démontrer la distribution des blocs HDFS.

```

      +-----+
      |  NameNode  |
      +-----+
          |
    +-----+-----+
    |             |             |
+-----+ +-----+ +-----+
|  DataNode 1  | |  DataNode 2  | |  DataNode 3  |
+-----+ +-----+ +-----+

      +-----+
      | Spark Notebook |
      +-----+
```

5. Structure du projet

```
streaming-analysis/  
  
├── data/  
│   ├── abannes.csv  
│   └── abannes_cluster_demo.csv  
├── notebooks/  
│   └── analyse_abannes.ipynb  
├── scripts/  
│   ├── upload_hdfs.sh  
│   ├── upload_hdfs_cluster.sh  
│   └── magnify.py  
├── screenshots/  
│   ├── hdfs_file.png  
│   ├── datanodes.png  
│   ├── blocks_distribution.png  
│   └── spark_results.png  
├── docker-compose.yml  
├── docker-compose-cluster.yml  
└── README.md
```

6. Création de l'environnement

6.1 Création de l'environnement simple

L'environnement simple contient :

- 1 NameNode ;
- 1 DataNode ;
- 1 conteneur Jupyter/PySpark.

Lancement :

```
docker compose up -d
```

Vérification des conteneurs :

```
docker ps
```

Résultat attendu :

```
namenode  
datanode  
spark-notebook
```

6.2 Création de l'environnement cluster

L'environnement cluster contient :

- 1 NameNode ;
- 3 DataNodes ;
- 1 conteneur Jupyter/PySpark.

Lancement :

```
docker compose -f docker-compose-cluster.yml up -d
```

Vérification :

```
docker ps
```

Vérification des DataNodes :

```
docker exec namenode hdfs dfsadmin -report
```

Résultat attendu :

```
Live datanodes (3)
```

Cela montre que les trois DataNodes sont bien connectés au NameNode.

7. Importation du fichier dans HDFS

7.1 Création du dossier HDFS

```
docker exec namenode hdfs dfs -mkdir -p /data
```

Cette commande crée un dossier `/data` dans HDFS.

7.2 Copie du fichier CSV vers le conteneur NameNode

```
docker cp ./data/abonnes.csv namenode:/tmp/abonnes.csv
```

Cette commande copie le fichier local `abonnes.csv` vers le conteneur `namenode`.

7.3 Importation du fichier dans HDFS

```
docker exec namenode hdfs dfs -put -f /tmp/abonnes.csv /data/
```

Cette commande importe le fichier dans HDFS.

7.4 Vérification de l'importation

```
docker exec namenode hdfs dfs -ls /data
```

Résultat attendu :

```
/data/abonnes.csv
```

8. Création du Notebook Spark

Un notebook Jupyter nommé `analyse_abonnes.ipynb` a été créé dans le dossier `notebooks/`.

Ce notebook contient :

- la création de la SparkSession ;
- le chargement des données depuis HDFS ;
- l'affichage des données ;
- les requêtes Spark demandées ;
- les résultats des analyses.

Accès à Jupyter :

```
http://localhost:8888
```

9. Configuration de l'environnement Spark

Dans le notebook, Spark est initialisé avec le code suivant :

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import sum, avg, count, col, round

spark = SparkSession.builder
    .appName("Analyse Abonnes")
    .getOrCreate()
```

Cette étape permet de créer une session Spark afin d'exécuter les traitements distribués.

10. Chargement des données depuis HDFS

Le fichier `abonnes.csv` est chargé depuis HDFS avec la commande suivante :

```
df = spark.read.csv(
    "hdfs://namenode:9000/data/abonnes.csv",
    header=True,
    inferSchema=True
)
```

Affichage des premières lignes :

```
df.show()
```

Affichage du schéma :

```
df.printSchema()
```

Le schéma attendu contient les colonnes suivantes :

```
id_abonne  
ville  
age  
abonnement  
duree_utilisation_mois  
montant
```

11. Liste des commandes Hadoop utilisées

Création du dossier HDFS

```
docker exec namenode hdfs dfs -mkdir -p /data
```

Copie du fichier dans le conteneur NameNode

```
docker cp ./data/abonnes.csv namenode:/tmp/abonnes.csv
```

Importation du fichier dans HDFS

```
docker exec namenode hdfs dfs -put -f /tmp/abonnes.csv /data/
```

Liste des fichiers dans HDFS

```
docker exec namenode hdfs dfs -ls /data
```


Affichage du contenu du fichier

```
docker exec namenode hdfs dfs -head /data/abonnes.csv
```

Vérification des blocs HDFS

```
docker exec namenode hdfs fsck /data/abonnes.csv -files -blocks -locations
```

Rapport des DataNodes

```
docker exec namenode hdfs dfsadmin -report
```

12. Liste des requêtes Spark utilisées

12.1 Nombre total d'abonnés

```
total_abonnes = df.count()
print(f"Nombre total d'abonnés : {total_abonnes}")
```

Cette requête permet de calculer le nombre total d'abonnés présents dans le fichier.

12.2 Revenu total

```
df.agg(
    round(sum("montant"), 2).alias("revenu_total")
).show()
```

Cette requête calcule la somme totale des revenus générés par les abonnés.

12.3 Revenu par type d'abonnement

```
df.groupBy("abonnement")
    .agg(round(sum("montant"), 2).alias("revenu_total"))
```

```
.orderBy(col("revenu_total").desc())  
.show()
```

Cette requête regroupe les abonnés par type d'abonnement et calcule le revenu total pour chaque catégorie.

12.4 Âge moyen des abonnés

```
df.agg(  
    round(avg("age"), 2).alias("age_moyen")  
) .show()
```

Cette requête calcule l'âge moyen des abonnés.

12.5 Nombre d'abonnés par ville

```
df.groupBy("ville")  
    .agg(count("*").alias("nombre_abonnes"))  
    .orderBy(col("nombre_abonnes").desc())  
    .show()
```

Cette requête calcule le nombre d'abonnés dans chaque ville.

12.6 Ville la plus représentée

```
df.groupBy("ville")  
    .agg(count("*").alias("nombre_abonnes"))  
    .orderBy(col("nombre_abonnes").desc())  
    .limit(1)  
    .show()
```

Cette requête permet d'identifier la ville contenant le plus grand nombre d'abonnés.

12.7 Durée moyenne d'utilisation

```
df.agg(  
    round(avg("duree_utilisation_mois"), 2).alias("duree_moyenne_mois")  
).show()
```

Cette requête calcule la durée moyenne d'utilisation des abonnés.

13. Résultats obtenus

Les résultats obtenus dans le notebook permettent de répondre aux questions demandées :

1. Nombre total d'abonnés.
2. Revenu total.
3. Revenu par type d'abonnement.
4. Âge moyen des abonnés.
5. Nombre d'abonnés par ville.
6. Ville la plus représentée.
7. Durée moyenne d'utilisation.

Les captures d'écran des résultats Spark doivent être ajoutées dans cette partie du rapport PDF.

Emplacements recommandés :

```
screenshots/spark_results.png  
screenshots/hdfs_file.png  
screenshots/datanodes.png  
screenshots/blocks_distribution.png
```

14. Création d'un fichier volumineux avec magnify.py

Le fichier original `abonnes.csv` étant trop petit pour démontrer efficacement la distribution des blocs HDFS, un script Python nommé `magnify.py` a été utilisé.

Ce script permet de générer un fichier plus volumineux en dupliquant les lignes du fichier original tout en conservant la même structure.

Objectif du script

L'objectif de `magnify.py` est de créer un fichier :

```
abonnes_cluster_demo.csv
```

Ce fichier sert uniquement à démontrer la répartition des blocs HDFS sur plusieurs DataNodes.

Exemple de logique du script

```
import pandas as pd

df = pd.read_csv("data/abonnes.csv")

big_df = pd.concat([df] * 70, ignore_index=True)

big_df["id_abonne"] = range(1, len(big_df) + 1)

big_df.to_csv("data/abonnes_cluster_demo.csv", index=False)
```

Le fichier généré atteint environ :

```
22 Mo
```

Cela permet à HDFS de découper le fichier en plusieurs blocs.

15. Importation du fichier de démonstration cluster

Le fichier `abonnes_cluster_demo.csv` a été importé dans HDFS avec une taille de bloc de 1 Mo.

```
docker exec namenode hdfs dfs -mkdir -p /data

docker cp ./data/abonnes_cluster_demo.csv namenode:/tmp/abonnes_cluster_demo.csv

docker exec namenode hdfs dfs
-D dfs.blocksize=1048576
```

```
-D dfs.replication=1  
-put -f /tmp/abonnes_cluster_demo.csv /data/
```

Explication :

- `dfs.blocksize=1048576` : taille de bloc fixée à 1 Mo.
- `dfs.replication=1` : une seule copie de chaque bloc afin de mieux observer la distribution.
- `abonnes_cluster_demo.csv` : fichier volumineux utilisé pour la démonstration.

16. Démonstration du stockage distribué

Pour visualiser la distribution des blocs du fichier volumineux :

```
docker exec namenode hdfs fsck  
/data/abonnes_cluster_demo.csv  
-files  
-blocks  
-locations
```

Cette commande affiche :

- le nombre de blocs ;
- l'emplacement de chaque bloc ;
- les DataNodes utilisés.

Exemple attendu :

```
Block 0 -> datanode1  
Block 1 -> datanode2  
Block 2 -> datanode3  
Block 3 -> datanode1  
...
```

Cette étape démontre que le fichier volumineux est bien réparti entre plusieurs DataNodes du cluster Hadoop.

17. Lecture du fichier cluster avec Spark

Pour vérifier que Spark peut lire le fichier distribué :

```
df_big = spark.read.csv(  
    "hdfs://namenode:9000/data/abonnes_cluster_demo.csv",  
    header=True,  
    inferSchema=True  
)  
  
df_big.count()
```

Cette commande permet de vérifier que Spark peut lire le fichier volumineux stocké dans HDFS.

18. Difficultés rencontrées

Plusieurs difficultés ont été rencontrées pendant la réalisation du projet :

Permissions Docker et Jupyter

Au départ, certains dossiers montés dans Docker ne permettaient pas la création ou la modification de fichiers depuis Jupyter.

Solution :

```
sudo chown -R 1000:1000 notebooks data scripts screenshots  
chmod -R u+rwX notebooks data scripts screenshots
```

Problème d'authentification Jupyter

Jupyter demandait un token d'accès.

Solution :

- utiliser l'URL avec le token depuis les logs Docker ;
- ou définir `JUPYTER_TOKEN=` dans le fichier Docker Compose.

Taille insuffisante du fichier original

Le fichier `abonnes.csv` était trop petit pour être divisé en plusieurs blocs visibles.

Solution :

- création du script `magnify.py` ;
- génération de `abonnes_cluster_demo.csv`.

Taille minimale des blocs HDFS

Une erreur est apparue lors de l'utilisation d'une taille de bloc de 64 Ko :

```
Specified block size is less than configured minimum value  
65536 < 1048576
```

Solution :

- utiliser une taille de bloc minimale de 1 Mo :

```
-D dfs.blocksize=1048576
```

19. Captures d'écran à insérer dans le rapport PDF

Le rapport PDF doit contenir les captures suivantes :

1. Conteneurs Docker lancés.
2. Interface HDFS sur `localhost:9870`.
3. Fichier `abonnes.csv` visible dans HDFS.
4. Liste des DataNodes.
5. Résultat de la commande `fsck`.
6. Notebook Jupyter.
7. Chargement du fichier depuis HDFS.
8. Résultats des requêtes Spark.
9. Distribution des blocs du fichier `abonnes_cluster_demo.csv`.

20. Conclusion

Ce projet a permis de mettre en place une architecture Big Data complète basée sur Hadoop HDFS et Apache Spark.

Le fichier `abonnes.csv` a été importé dans HDFS puis analysé avec Spark à travers un Notebook Jupyter. Les requêtes Spark ont permis d'obtenir les indicateurs demandés : nombre total d'abonnés, revenu total, revenu par abonnement, âge moyen, nombre d'abonnés par ville, ville la plus représentée et durée moyenne d'utilisation.

Une démonstration complémentaire a été réalisée avec un cluster Hadoop composé d'un NameNode et de trois DataNodes. Comme le fichier original était trop petit pour montrer une vraie distribution des blocs, le

script `magnify.py` a été utilisé afin de générer un fichier plus volumineux. Ce fichier a permis d'observer la répartition des blocs HDFS sur plusieurs DataNodes.

Ce projet a donc permis de comprendre concrètement :

- le rôle de HDFS dans le stockage distribué ;
- le rôle du NameNode et des DataNodes ;
- l'utilisation de Spark pour l'analyse de données ;
- l'intérêt d'un cluster Hadoop pour traiter de plus grands volumes de données.